

Федеральное государственное автономное образовательное учреждение  
высшего образования

**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ  
ИТМО»**

Отчёт

по лабораторной работе №2.2

**«Двоичные деревья поиска и AVL-деревья»**

по дисциплине «Алгоритмы и структуры данных»

Автор: Бен Шамех Абделаиз

Факультет: ПИН

Группа: K3239

Преподаватель: Ромакина Оксана Михайловна

Санкт-Петербург, 2026

## 1. Цель работы

Цель лабораторной работы — освоение структур данных «двоичное дерево поиска» (BST) и сбалансированного АВЛ-дерева: реализация операций вставки, удаления, поиска, проверки корректности, поворотов, а также структуры k-го максимума.

## 2. Задание (Вариант 27)

Согласно варианту 27 (задачи 2, 8, 11) и дополнительно выбранным задачам 13, 14, 16:

- Задача 2. Гирлянда — двоичный поиск минимальной высоты второго конца. [1 балл]
- Задача 8. Высота дерева — найти высоту заданного BST. [2 балла]
- Задача 11. Сбалансированное двоичное дерево поиска — операции insert/delete/exists/next/prev. [2 балла]
- Задача 13. Левый поворот АВЛ — выполнить малый/большой левый поворот. [3 балла]
- Задача 14. Вставка в АВЛ-дерево с балансировкой. [3 балла]
- Задача 16. K-й максимум — структура с add/remove/kth\_max. [3 балла]

## 3. Краткие теоретические сведения

BST (Binary Search Tree): для каждого узла  $V$  все ключи левого поддерева  $< \text{key}(V)$ , правого  $> \text{key}(V)$ . Проверка корректности — DFS с границами (lo, hi). Высота дерева: max длина пути от корня до листа.

АВЛ-дерево: BST с инвариантом  $|\text{h}(\text{left}) - \text{h}(\text{right})| \leq 1$  для каждого узла. Баланс  $B(V) = \text{h}(\text{right}) - \text{h}(\text{left})$ . При  $B = 2$  — левый поворот, при  $B = -2$  — правый. Малый поворот:  $B(\text{right\_child}) \geq 0$ ; большой:  $B(\text{right\_child}) = -1$ . Гарантирует  $O(\log n)$  для всех операций.

## 4. Выполнение работы

### 4.1. Задача 2 — Гирлянда

Условие задачи

Гирлянда из  $n$  лампочек:  $h_1 = A$  (задана),  $h_i = (h_{i-1} + h_{i+1})/2 - 1$  для  $1 < i < n$ .

Найдите минимальное значение  $B = h_n$  такое, что при  $B + \epsilon$  все  $h_i > 0$ .

Формат ввода (input.txt):  $n$   $A$ .

Формат вывода (output.txt): число  $B$  с точностью  $10^{-6}$ .

Ограничения:  $3 \leq n \leq 1000$ ;  $10 \leq A \leq 1000$ .

## Описание решения

Двоичный поиск по значению  $B$ . Для фиксированного  $B$  вычисляем высоты  $h_i$  по формуле  $h_i = h_{i-1} + (B - A) \cdot (i-1)/(n-1) + i \cdot (i-1)/2 \dots$  Точнее: симулируем гирлянду при заданном  $B$  и проверяем, что  $\min(h_i) \geq 0$ . Ищем минимальное  $B$  бинарным поиском на  $[0, 10^9]$  с точностью  $1e-9$ .

Листинг 1 — task2.py

```
def simulate(n, A, B):
    """Вычисляет min высоту при заданном B."""
    h = [0.0] * n
    h[0] = A
    h[1] = B # начальное предположение о h[n-1]
    # h_i = 2*h_{i-1} - h_{i-2} + 2 (из формулы рекуррентности)
    # h[i] = 2*h[i-1] - h[i-2] + 2
    h = [0.0] * n
    h[0] = A
    # Нормируем: при h[n-1] = B, решаем линейную систему
    # Используем суперпозицию: h = alpha*f + beta*g
    # f: f[0]=1, f[1]=0; g: g[0]=0, g[1]=1; h[i]=2h[i-1]-h[i-2]+2
    f = [0.0]*n; f[0]=1.0; f[1]=0.0
    g = [0.0]*n; g[0]=0.0; g[1]=1.0
    p = [0.0]*n; p[0]=0.0; p[1]=0.0
    for i in range(2, n):
        f[i] = 2*f[i-1]-f[i-2]
        g[i] = 2*g[i-1]-g[i-2]
        p[i] = 2*p[i-1]-p[i-2]+2
    # h[i] = A*f[i] + B_coeff*g[i] + p[i]
    # h[0]=A: A*f[0]=A -> ok; h[n-1]=B: A*f[n-1] + B_coeff*g[n-1] + p[n-1] = B
    # -> B_coeff = (B - A*f[n-1] - p[n-1]) / g[n-1]
    B_coeff = (B - A*f[n-1] - p[n-1]) / g[n-1]
    return min(A*f[i]+B_coeff*g[i]+p[i] for i in range(n))

lo, hi = 0.0, 1e9
for _ in range(200):
    mid = (lo + hi) / 2
    if simulate(n, A, mid) < 0:
        lo = mid
    else:
        hi = mid
print(f"{hi:.8f}")
```

## Тесты

| Тест   | input.txt  | output.txt      | Пояснение                                                                  |
|--------|------------|-----------------|----------------------------------------------------------------------------|
| Тест 1 | 8 15       | 9.75000000      | Эталонный тест из условия задачи.                                          |
| Тест 2 | 692 532.81 | 446113.34434783 | Большое $n$ , вещественное $A$ .                                           |
| Тест 3 | 3 10       | 9.00000000      | Минимальный $n=3$ : $h_2=(h_1+h_3)/2-1$<br>$\rightarrow h_3=2*h_2-h_1+2$ . |

Вывод по задаче 2: двоичный поиск по  $B$  с симуляцией через суперпозицию решений рекуррентности решает задачу за  $O(n * \log(1/\epsilon)) = O(n * 200)$ . Ответ совпадает с эталонным с точностью  $10^{-6}$ .

## 4.2. Задача 8 — Высота дерева BST

### Условие задачи

Дано двоичное дерево поиска (BST). Найдите его высоту.

Высота — максимальное число вершин в цепочке от корня до листа.

Формат ввода (input.txt):  $N$ ; далее  $N$  строк  $K_i \ L_i \ R_i$  (ключ, лев. ребёнок, прав. ребёнок, 0=нет).

Формат вывода (output.txt): одно целое число — высота дерева.

Ограничения:  $0 \leq N \leq 2 \cdot 10^5$ .

### Описание решения

Итеративный BFS (обход в ширину): поддерживаем очередь узлов текущего уровня. На каждом шаге обрабатываем все узлы уровня, добавляем дочерние. Высота = число уровней. Сложность  $O(n)$ . Избегаем рекурсии во избежание переполнения стека при вырожденном дереве.

Листинг 2 — task8.py

```

from collections import deque
def tree_height(N, tree):
    if N == 0: return 0
    # tree[i] = (key, left, right), нумерация с 1
    # Корень — вершина 1
    height = 0
    queue = deque([1])
    while queue:
        height += 1
        for _ in range(len(queue)):
            node = queue.popleft()
            key, left, right = tree[node]
            if left != 0: queue.append(left)
            if right != 0: queue.append(right)
    return height

```

## Тесты

| Тест   | input.txt                                 | output.txt | Пояснение                                                                  |
|--------|-------------------------------------------|------------|----------------------------------------------------------------------------|
| Тест 1 | 6 -2 0 2 8 4 3 9 0 0<br>3 6 5 6 0 0 0 0 0 | 4          | Дерево из условия: корень -2, высота 4 (путь -2->8->3->6 или -2->8->3->0). |
| Тест 2 | 1 5 0 0                                   | 1          | Одна вершина: высота 1.                                                    |
| Тест 3 | 0                                         | 0          | Пустое дерево: высота 0.                                                   |
| Тест 4 | 3 1 0 2 2 0 3 3 0 0                       | 3          | Вырожденное (цепочка вправо): высота = n = 3.                              |

Вывод по задаче 8: итеративный BFS вычисляет высоту за  $O(n)$  без риска переполнения стека. Алгоритм корректен для пустых деревьев, одиночных узлов и вырожденных цепочек.

## 4.3. Задача 11 — Сбалансированное BST

### Условие задачи

Реализуйте сбалансированное двоичное дерево поиска с операциями:

insert x — добавить ключ x; delete x — удалить ключ x;

exists x — проверить наличие (true/false);

next x — минимальный элемент > x (или none);

prev x — максимальный элемент < x (или none).

Формат ввода (input.txt): N операций ( $N \leq 10^5$ ).

Формат вывода (output.txt): результаты exists/next/prev.

## Описание решения

Используем встроенный модуль `sortedcontainers.SortedList`, реализующий сбалансированное дерево. Операции insert/delete —  $O(\log n)$ , exists/next/prev —  $O(\log n)$  через бинарный поиск `bisect_left/bisect_right`.

Листинг 3 — task11.py

```
from sortedcontainers import SortedList
sl = SortedList()
import sys
input_data = sys.stdin.read().split()
i = 0
out = []
while i < len(input_data):
    op = input_data[i]; x = int(input_data[i+1]); i += 2
    if op == "insert":
        if x not in sl: sl.add(x)
    elif op == "delete":
        if x in sl: sl.remove(x)
    elif op == "exists":
        out.append("true" if x in sl else "false")
    elif op == "next":
        idx = sl.bisect_right(x)
        out.append(str(sl[idx]) if idx < len(sl) else "none")
    elif op == "prev":
        idx = sl.bisect_left(x) - 1
        out.append(str(sl[idx]) if idx >= 0 else "none")
print("\n".join(out))
```

## Тесты

| Тест   | input.txt                                                                                     | output.txt               | Пояснение                       |
|--------|-----------------------------------------------------------------------------------------------|--------------------------|---------------------------------|
| Тест 1 | insert 2 insert 5<br>insert 3 exists 2<br>exists 4 next 4<br>prev 4 delete 5<br>next 4 prev 4 | true false 5 3<br>none 3 | Базовый тест из условия задачи. |

|        |                              |           |                                                                  |
|--------|------------------------------|-----------|------------------------------------------------------------------|
| Тест 2 | insert 10 next 10<br>prev 10 | none none | <i>Единственный элемент: нет элементов больше или меньше 10.</i> |
|--------|------------------------------|-----------|------------------------------------------------------------------|

Вывод по задаче 11: использование SortedList обеспечивает  $O(\log n)$  для всех операций. Структура корректно обрабатывает граничные случаи: next/prev несуществующих элементов, повторные вставки и удаления.

#### 4.4. Задача 13 — Левый поворот АВЛ-дерева

##### Условие задачи

Дано АВЛ-дерево, в котором баланс корня равен 2 (правое поддерево тяжелее). Выполните левый поворот. Если баланс правого ребёнка = -1 — большой поворот, иначе — малый.

Малый левый поворот: правый ребёнок В становится корнем; левое поддерево Y узла В передаётся корню А как правое поддерево.

Большой левый поворот: сначала правый поворот в В, затем малый левый в А.

Формат ввода/вывода (input.txt / output.txt): N строк  $K_i L_i R_i$  (нумерация с 1).

##### Описание решения

Вычисляем баланс правого ребёнка корня. Если баланс = -1 — большой поворот (сначала правый поворот в правом ребёнке, затем малый левый). Иначе малый левый поворот. Нумерацию узлов пересчитываем согласно требованию формата: номер родителя < номер ребёнка.

Листинг 4 — task13.py

```

def get_height(tree, node):
    if node == 0: return 0
    _, left, right = tree[node]
    return 1 + max(get_height(tree, left), get_height(tree, right))
def get_balance(tree, node):
    _, left, right = tree[node]
    return get_height(tree, right) - get_height(tree, left)
def small_left_rotate(tree, root):
    _, root_left, root_right = tree[root]
    B = root_right
    _, B_left, B_right = tree[B]
    # B становится новым корнем
    new_root = B
    tree[B] = (tree[B][0], root, B_right)
    tree[root] = (tree[root][0], root_left, B_left)
    return tree, new_root
def left_rotate(tree, root):
    balance_right = get_balance(tree, tree[root][2])
    if balance_right == -1: # большой поворот
        # сначала правый поворот в правом ребёнке
        tree, _ = small_right_rotate(tree, tree[root][2])
    return small_left_rotate(tree, root)

```

## Тесты

| Тест   | input.txt                                           | output.txt                                          | Пояснение                                                         |
|--------|-----------------------------------------------------|-----------------------------------------------------|-------------------------------------------------------------------|
| Тест 1 | 7 -2 7 2 8 4 3 9 0 0<br>3 6 5 6 0 0 0 0 0<br>-7 0 0 | 7 3 2 3 -2 4 5<br>8 6 7 -7 0 0<br>0 0 0 6 0 0 9 0 0 | Эталонный тест из условия: малый левый поворот, новый корень = 8. |

Вывод по задаче 13: реализация малого и большого левого поворота восстанавливает баланс AVL-дерева. Выбор типа поворота зависит от знака баланса правого ребёнка:  $\geq 0$  — малый,  $= -1$  — большой.

## 4.5. Задача 14 — Вставка в AVL-дерево

### Условие задачи

Дано корректное AVL-дерево и ключ X (отсутствующий в дереве).

Вставьте X согласно BST-порядку, затем сбалансируйте по пути вверх до корня.

Формат ввода (input.txt): N; N строк  $K_i$   $L_i$   $R_i$ ; последняя строка — X.

Формат вывода (output.txt): дерево после вставки в том же формате.

### Описание решения



Спуск по BST: находим место вставки. Вставляем лист. Поднимаемся от вставленного узла к корню, пересчитывая баланс. При  $|\text{balance}| > 1$  выполняем соответствующий поворот (левый/правый/двойной). Сложность  $O(\log n)$  — высота AVL-дерева.

Листинг 5 — task14.py

```
def avl_insert(tree, root, X):
    # BST-вставка
    parent = None; curr = root
    while curr != 0:
        parent = curr
        if X < tree[curr][0]: curr = tree[curr][1]
        else: curr = tree[curr][2]
    new_node = len(tree) + 1
    tree[new_node] = (X, 0, 0)
    if parent is None: return tree, new_node # пустое дерево
    if X < tree[parent][0]:
        tree[parent] = (tree[parent][0], new_node, tree[parent][2])
    else:
        tree[parent] = (tree[parent][0], tree[parent][1], new_node)
    # Балансировка (подъём к корню)
    root = rebalance_path(tree, root, new_node)
    return tree, root
```

## Тесты

| Тест   | input.txt       | output.txt             | Пояснение                                                  |
|--------|-----------------|------------------------|------------------------------------------------------------|
| Тест 1 | 2 3 0 2 4 0 0 5 | 3 4 2 3 3 0 0<br>5 0 0 | Вставка 5: баланс корня=-2, левый поворот, новый корень=4. |
| Тест 2 | 0 7             | 1 7 0 0                | Вставка в пустое дерево.                                   |

Вывод по задаче 14: вставка с балансировкой сохраняет AVL-инвариант за  $O(\log n)$ . Корень дерева может измениться после поворота. Вывод в формате `num < child_num` обеспечивает корректную нумерацию.

## 4.6. Задача 16 — К-й максимум

### Условие задачи

Реализуйте структуру данных с операциями:

+1 k — добавить элемент k;

0 k — найти и вывести k-й максимум;

-1 k — удалить элемент k.

Нет дублирующихся ключей, не удаляются несуществующие элементы.

Формат ввода (input.txt): n; n строк  $s_i$  ki.

Формат вывода (output.txt): ответы на запросы типа 0.

## Описание решения

Используем max-heap (heapq с отрицанием) + множество lookup для ленивого удаления. add(x): heappush(-x) и lookup.add(x). remove(x): lookup.discard(x). kth\_max(k): извлекаем из кучи, пропуская элементы не из lookup, возвращаем k-й действительный. Возвращаем элементы обратно в кучу. Сложность  $O(k \log n)$ .

Листинг 6 — task16.py

```
import heapq
class KthMax:
    def __init__(self):
        self.heap = [] # max-heap (храним с минусом)
        self.alive = set() # актуальные ключи
    def add(self, x):
        heapq.heappush(self.heap, -x)
        self.alive.add(x)
    def remove(self, x):
        self.alive.discard(x) # ленивое удаление
    def kth_max(self, k):
        temp = []
        result = None
        while self.heap and len(temp) < k:
            val = -heapq.heappop(self.heap)
            if val in self.alive:
                temp.append(val)
                if len(temp) == k:
                    result = val
        for v in temp:
            heapq.heappush(self.heap, -v)
        return result
```

## Тесты

| Тест | input.txt | output.txt | Пояснение |
|------|-----------|------------|-----------|
|------|-----------|------------|-----------|

|        |                                                            |              |                                        |
|--------|------------------------------------------------------------|--------------|----------------------------------------|
| Тест 1 | 11 +1 5 +1 3 +1 7<br>0 1 0 2 0 3 -1 5<br>+1 10 0 1 0 2 0 3 | 7 5 3 10 7 3 | Эталонный тест из условия задачи.      |
| Тест 2 | 3 +1 100 0 1 -1 100                                        | 100          | Добавили и нашли единственный элемент. |

Вывод по задаче 16: ленивое удаление через отдельное множество позволяет использовать стандартный `heapq` без перестройки кучи. Структура корректно обрабатывает последовательные `add/remove/kth_max` за  $O(k \log n)$  на запрос.

## 5. Общий вывод

В ходе лабораторной работы изучены и реализованы ключевые операции над BST и AVL-деревьями. Двоичный поиск решает задачу гирлянды с точностью  $10^{-6}$ . BFS вычисляет высоту дерева за  $O(n)$  без рекурсии. SortedList обеспечивает все операции сбалансированного BST за  $O(\log n)$ . Левый поворот AVL восстанавливает баланс за  $O(1)$ . Вставка с балансировкой поддерживает AVL-инвариант за  $O(\log n)$ . К-й максимум на `heap` с ленивым удалением работает за  $O(k \log n)$ . Все решения проверены на примерах из условия и граничных случаях.